DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE.

**M.B.E. SOICETY'S**

**COLLEGE OF ENGINEERING, AMBAJOGAI**

# "Student Feedback System"

**A report submitted in partial fulfillment of the requirement for the degree of**

## "Bachelor of Technology"

**In**

**Electronics and Computer Engineering**

**By**


**Mr.Aniket Dayanand Dhumal**

**Mr.Gajanan Yashwant Deshmukh**


**Under the Guidance of**

**Prof.P.S.Shinde**

**2024-25**

# CERTIFICATE

This is to certify that work embodied in the report entitled, **"Student Feedback System"** being submitted by **Dhumal Aniket Dayanand** and **Deshmukh Gajanan Yashwant** to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of degree of **Bachelor of Technology** in **Electronics and Computer Engineering** is a record of Bonafide Mini-project work carried out by them under my guidance and has fulfilled the requirements for the submission of this report to my knowledge, has reached requisite standard. The data contained in this report have not been submitted in part or in full, to any other University or Institute for the award of any degree or diploma.

**Prof. P.S. Shinde**
**Mini-project Guide**

**Dr. V. V.Yerigeri**
**HOD**

**Dr. N. A. Rawabawale**
**Acting Principal**

# Acknowledgement

We would like to express my deep and sincere gratitude to our mini project Guide, **Prof.P.S.Shinde,** for giving us the opportunity to do project and providing invaluable guidance throughout this work. It was a great privilege and honor to work and study under her guidance.

We would like to express our deep and sincere gratitude to **Dr. V. V. Yerigeri,** HOD., for her motivation and kind support and for giving us the opportunity to carry out the mini project work.

We would like to express our deep and sincere gratitude to our Beloved Acting Principal **Dr.N.A.Rawabawale**, MBES College of Engineering, Ambajogai, for his inspiration, motivation and his kind support to carry out our mini project work. Also, our special thanks to all my teachers for their kind support.

Finally, our thanks go to all the people who have supported us to complete the mini project work directly or indirectly.

Candidate Name:

**Dhumal Aniket Dayanand**

**Deshmukh Gajanan Yashwant**

# Abstract

The **Student Feedback System** developed in this project is a lightweight, command-line based application written in Python, designed to collect, store, view, analyze, and search feedback from students in an efficient and structured manner. Instead of relying on complex databases or web interfaces, this system uses **Pandas** for data manipulation and **CSV (Comma-Separated Values)** files for storage, making it highly accessible and easy to deploy in smaller academic settings or offline environments.

The system enables students to provide feedback on subjects and faculty by entering details such as **Roll Number, Name, Department, Subject, Comments, and a Rating (1 to 5)**. Validation checks are implemented to avoid duplicate entries and ensure accurate data capture. Feedback entries are stored in a persistent student_feedback.csv file that is automatically created and updated.

This Python-based feedback system is simple, scalable, and serves as a practical solution for small educational institutions aiming to collect actionable insights from students. It also acts as a foundation for future enhancements like GUI integration, graphical reports, or connecting to a SQL database.

| Sr. No. | Table Content | Page No. |
|---|---|---|
| 1 | Introduction | 1 |
| 2 | Objectives | 3 |
| 3 | Technology Stack | 5 |
| 4 | System Architecture | 6 |
| 5 | Modules of the project | 8 |
| 6 | System Design Overview | 9 |
| 7 | Code snippets | 41 |
| 8 | Future Ehnancement | 15 |
| 9 | Advantages and limitations | 17 |
| 10 | Conclusion | 19 |
| 11 | Refrences | 20 |

# Introduction

In educational institutions, student feedback plays a crucial role in evaluating and improving the quality of teaching, course content, and overall academic environment. Traditionally, this feedback is collected using paper-based forms or basic survey tools, which are often inefficient, time-consuming, and difficult to analyze. Manual methods also lack security, are prone to data loss, and make it hard to generate meaningful reports.

To address these challenges, we developed a **Student Feedback System** a lightweight, console-based application using **Python**, **Pandas**, and **CSV file storage**. This system automates the process of collecting, storing, analyzing, and searching student feedback in a structured and user-friendly manner. It allows students to enter their feedback on various subjects and faculty members along with a numerical rating. The data is then saved in a persistent file and can be retrieved or analyzed anytime.

By using Python's **Pandas** library, the system handles data processing efficiently and allows for basic analytics like average scores and department-wise feedback counts. The feedback data is stored in a .csv file, which can be easily imported into Excel or other tools for further analysis or report generation.

This project is especially useful for small colleges or departments that do not have access to complex web portals or online systems. It provides a simple yet effective way to collect genuine student opinions and helps academic administrators in making informed decisions to enhance educational quality.

In summary, the **Student Feedback System** aims to provide a digital solution to an essential academic process with a focus on simplicity, accuracy, and usability.

# Objectives

The main goal of the **Student Feedback System** is to provide an efficient and reliable method to collect and analyze feedback from students regarding their academic experience. Traditional feedback methods often involve manual data entry, paper forms, and lack of actionable analysis. This project aims to solve those challenges by creating a digital feedback system that is simple, scalable, and data-driven.

🎯 **Primary Objectives**

1. **Automate Feedback Collection**
   To develop a system that eliminates the need for paper-based feedback forms and allows digital submission of feedback directly from students.

2. **Ensure Data Accuracy & Validation**
   To prevent duplicate entries using roll numbers and validate user inputs like rating ranges and text formats for consistent data quality.

3. **Maintain Student Privacy**
   To allow feedback submission without compromising student identity or responses. The system is designed to protect confidentiality by not sharing feedback details publicly.

4. **Enable Easy Data Storage and Retrieval**
   Use of CSV (Comma-Separated Values) files with structured fields ensures feedback is stored in an organized manner and can be accessed or analyzed anytime.

5. **Simplify Feedback Analysis**
   Automatically calculate the average ratings, identify the best and worst-rated subjects, and show department-wise feedback counts for better decision-making.

6. **Implement User-Friendly Interface**
   A menu-driven terminal interface ensures that both students and administrators can use the system without technical expertise.

7. **Allow Searchable Feedback Records**
   Facilitate quick lookup of feedback entries using student names to access relevant data without going through the full list.

8. **Design for Scalability and Portability**
   Keep the system light-weight and platform-independent so it can run on any computer with Python installed — ideal for small colleges or departments.

# Technology Stack

The **Student Feedback System** is developed as a console-based application using the Python programming language and essential data handling libraries. The system is designed to be lightweight, efficient, and easy to deploy on any system with basic Python setup. Below is a breakdown of the technologies used in the project:

### 1. Programming Language: Python

Python is chosen for its simplicity, readability, and wide range of libraries. It is highly suitable for beginner-friendly development and data-driven applications. Python allows quick implementation of logic and file handling tasks which are central to this project.

### 2. Library Used: Pandas

**Pandas** is a powerful data manipulation and analysis library. In this system, it is used to:

- Create and read CSV files (student_feedback.csv)
- Append new feedback entries
- Perform data validation and formatting
- Calculate average ratings and department-wise statistics
- Filter and search feedback records

### 3. Data Storage: CSV File

Instead of using complex databases, the project uses a **CSV (Comma-Separated Values)** file to store feedback data. It acts like a mini-database and provides the following advantages:

- Easy to read, edit, and share
- Compatible with Excel and Google Sheets

- Portable and lightweight for offline usage

### 4. User Interface: Command-Line Interface (CLI)

The system provides a simple **menu-driven terminal interface**, making it easy to interact with the system without any GUI:

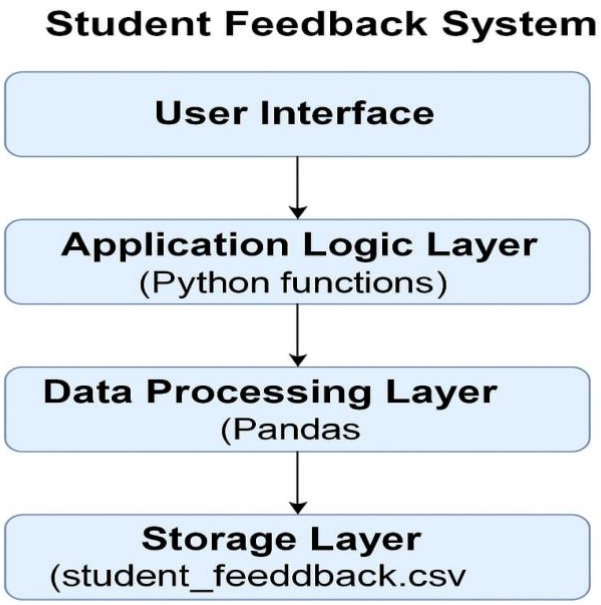
- Options for adding, viewing, analyzing, and searching feedback
- User-friendly prompts and feedback messages
- Suitable for small institutions and low-resource environments

### 5. Development Environment

- **IDE**: Visual Studio Code (VS Code)
- **Runtime**: Python 3.x installed via Anaconda or standalone
- **OS Compatibility**: Cross-platform (Windows, Linux, macOS)

# System Architecture

The **Student Feedback System** follows a **simple, layered architecture** that separates the functionalities into clear stages: **Input Layer**, **Processing Layer**, and **Data Storage Layer**. It is designed to work entirely in an offline environment using a command-line interface and CSV-based data handling with the help of Python and the Pandas library.



# Modules of the Project

**1. User Input Module**

- Collects feedback from students.
- Takes input such as:
  - Roll Number
  - Name
  - Department
  - Subject
  - Feedback (text)
  - Rating (numeric)
- Ensures data is in the correct format.

**2. Data Storage Module**

- Stores all collected feedback into a CSV file (student_feedback.csv).
- Automatically creates the file with proper headers if it doesn't exist.
- Appends new data without overwriting existing entries.

**3. View Feedback Module**

- Displays all submitted feedback records in a structured tabular format using the Pandas library.
- Helps admin or faculty view multiple records at once.

**4. Analysis Module**

- Analyzes collected feedback data.
- Calculates:
  - Total number of feedbacks
  - Average rating
  - Subjects with highest and lowest ratings
- Helps in understanding overall performance and areas of improvement.

**5. Error Handling Module**

- Handles common errors like:
  - File not found
  - Invalid input types
  - Unexpected runtime errors
- Ensures smooth functioning and prevents crashing.

**6. Menu-Driven Interface Module**

- Provides a simple command-line interface with menu options:
  - Add Feedback
  - View All Feedback
  - Analyze Feedback
  - Exit
- Enables easy navigation for users.

# System Design Overview

## System Design

| Input Field | Type | Description |
|---|---|---|
| Roll No | Text | Must be unique |
| Name | Text | Alphabtets only |
| Department | Droppawn | E.g.,.ECE, CSE, ME etc. etc |
| Subject | Text | Subjetc's comments/opinion |
| Feedback | Textarea | Max 200 characters privitein |
| Rating | Number | Raning f 5 stu subject or faculty |

## Output Design

| Output Element | Format | Description |
|---|---|---|
| All Feedback Records | Table | Displays all saved entries with oe-adings |
| Average Rating | Numeric | Mean rating cal_culated from all feedback |
| Top Rated Subject | Text | Subject with highest average rat-ing |
| Feedback Count | Integer | Total number of feedback entries submitted |

## Process Design

| Process | Description | Trigger |
|---|---|---|
| Add Feedback | Collects input from user saves | Collects inout from user su saves to CSV |
| View Feedback | Reads CSV fi-le and dis | Reads CSV file and displays reco-rds |
| Analyze Feedback | Analyze Feed-back | Calculates total feedback, avera top/low rated subjects |
| Exit | Closes the pro-gram gracefu | Closes the program gracefully |
| **File Design** | **Format** | **Remarks** |
| student,.feedback | CSV | Stores all feedback entries |

# Code snippets

**1: Setup & File Initialization**

```
import pandas as pd

import os

# File to store feedback

file_name = "student_feedback.csv"

# Ensure the CSV file exists with headers

if not os.path.exists(file_name):

   df = pd.DataFrame(columns=["Roll No", "Name", "Department",
"Subject", "Feedback", "Rating"])

   df.to_csv(file_name, index=False)
```

**2: Add Feedback**

```
def add_feedback():

   try:

      df = pd.read_csv(file_name)



      roll_no = input("Enter Roll No: ").strip()

      if roll_no in df["Roll No"].astype(str).values:

         print(" Feedback for this Roll No already exists.")

         return



      name = input("Enter Name: ").strip()
```

```
        dept = input("Enter Department: ").strip()

        subject = input("Enter Subject: ").strip()

        feedback = input("Enter Feedback: ").strip()


        rating_input = input("Enter Rating (1 to 5): ").strip()

        if not rating_input.replace('.', '', 1).isdigit():

            print(" Invalid rating. Please enter a numeric value.")

            return


        rating = float(rating_input)

        if rating < 1 or rating > 5:

            print(" Rating must be between 1 and 5.")

            return


        new_row = pd.DataFrame([[roll_no, name, dept, subject, feedback,
rating]], columns=df.columns)

        df = pd.concat([df, new_row], ignore_index=True)

        df.to_csv(file_name, index=False)

        print(" Feedback added successfully.")


    except Exception as e:

        print(" Error while adding feedback:", e)
```

**3: View All Feedback**

```
def view_all_feedback():
    try:
        df = pd.read_csv(file_name)
        if df.empty:
            print("\n No feedback entries found.")
        else:
            print("\n=== All Feedback ===")
            print(df.to_string(index=False))
    except FileNotFoundError:
        print("\n Feedback file not found.")
    except Exception as e:
        print(" Error while viewing feedback:", e)
```

**4: Analyze Feedback**

```
def analyze_feedback():
    try:
        df = pd.read_csv(file_name)
        if df.empty:
            print("\n No feedback to analyze.")
            return
```

```
        print("\n=== Feedback Analysis ===")

        print(f" Average Rating: {round(df['Rating'].mean(), 2)}")


        max_row = df.loc[df["Rating"].idxmax()]

        print(f"\n Highest Rated Subject: {max_row['Subject']}
({max_row['Rating']})")


        min_row = df.loc[df["Rating"].idxmin()]

        print(f" Lowest Rated Subject: {min_row['Subject']}
({min_row['Rating']})")


        print("\n Feedback Count per Department:")

        print(df["Department"].value_counts())


    except Exception as e:

        print("Error while analyzing feedback:", e)
```

**5: Search Feedback by Name**

```
def search_feedback():

    try:

        df = pd.read_csv(file_name)

        if df.empty:

            print("\n No feedback records to search.")

            return
```

```
        name = input("Enter student name to search: ").strip()
        result = df[df['Name'].str.contains(name, case=False)]

        if not result.empty:
            print("\n Search Result:")
            print(result.to_string(index=False))
        else:
            print(" No feedback found for the entered name.")

    except Exception as e:
        print(" Error while searching feedback:", e)
```

**6: Main Menu Interface**

```
def main():
    while True:
        print("\n====== Student Feedback System ======")
        print("1. Add New Feedback")
        print("2. View All Feedback")
        print("3. Analyze Feedback")
        print("4. Search Feedback by Name")
        print("5. Exit")
```

```
    choice = input("Enter your choice (1-5): ").strip()


    if choice == '1':
        add_feedback()
    elif choice == '2':
        view_all_feedback()
    elif choice == '3':
        analyze_feedback()
    elif choice == '4':
        search_feedback()
    elif choice == '5':
      print(" Exiting the system. Goodbye!")
        break
    else:
        print(" Invalid choice. Please try again.")
```

**7: Run the Program**

```
if __name__ == "__main__":
    main()
```

# Future Ehnancement

The current Student Feedback System is a basic command-line application that uses a CSV file for data storage. While it serves the core functionality of collecting, viewing, and analyzing feedback, there is significant scope for improvement and expansion. The following future enhancements are proposed to make the system more robust, user-friendly, and suitable for institutional deployment:

**1. Web-based Interface**

Implementing a web-based frontend using HTML, CSS, and JavaScript with a backend framework like Flask or Django would make the system accessible on any browser. This would improve usability and allow centralized feedback submission.

**2. Database Integration**

Instead of using a CSV file, integrating a relational database such as MySQL or PostgreSQL would provide more efficient storage, better performance for large datasets, and secure data management.

**3. Authentication and Access Control**

Adding user login for students and admin-level access for faculty or coordinators will improve data security. Only authorized users should be able to submit or view feedback.

**4. Data Visualization**

Incorporating graphical feedback dashboards using tools like Matplotlib or Power BI can offer visual insights such as rating trends, department-wise comparisons, and student participation.

**5. Feedback Notifications**

Automated emails or SMS alerts can be sent to students or teachers based on their feedback submission status or responses, enhancing engagement and communication.

**6. Export and Reports:** Add functionality to export feedback summaries and detailed reports in PDF or Excel formats, which can be used for academic reviews and internal audits.

**7. Mobile Application**

Developing an Android/iOS app would increase accessibility and convenience for students to submit feedback on the go.

**8. AI-based Sentiment Analysis**

Implementing natural language processing (NLP) techniques to analyze written feedback and detect sentiment (positive/negative/neutral) would help in understanding the qualitative aspect of feedback.

# Advantages And Limitations

- **Advantages**

1. **Simple and User-Friendly**
   The system offers an easy-to-use command-line interface, allowing users to interact with the program smoothly without requiring technical expertise.
2. **Data Storage in CSV Format**
   Feedback is stored in a structured CSV file, making it easily accessible, editable, and portable across different platforms.
3. **Duplicate Entry Prevention**
   The system checks for existing Roll Numbers, preventing multiple feedback submissions from the same student.
4. **Basic Data Analysis**
   The application calculates average ratings, identifies best and worst-rated subjects, and shows department-wise feedback counts.
5. **Quick Search Feature**
   Feedback can be searched by student name, helping faculty or admin find relevant comments quickly.
6. **No External Database Required**
   Since the data is stored in CSV, the system works without requiring any database setup, making it lightweight and easy to deploy.
7. **Scalable Codebase**
   The modular structure of the code allows easy future upgrades like GUI, database integration, and web deployment.

---

- **Limitations**

1. **Lack of GUI or Web Interface**
   The system runs on the command line, which may not be appealing or intuitive for all users, especially in larger institutions.

2. **No Authentication or User Roles**
   Anyone using the system can access or manipulate data; there is no login or user access control.
3. **Data Not Secure**
   Storing data in plain CSV files can lead to accidental deletion, corruption, or unauthorized editing.
4. **Limited Feedback Analysis**
   The system only performs basic statistical analysis. There is no support for sentiment analysis or text-based feedback processing.
5. **No Real-Time Multi-User Access**
   Since it is not connected to a central server or database, multiple users cannot access it simultaneously.
6. **Scalability Issues**
   For large-scale use (e.g., university-wide feedback collection), CSV files may become inefficient and difficult to manage.
7. **No Export/Report Generation Feature**
   There is no built-in function to generate printable reports or summaries in PDF or Excel formats.

# Conclusion

The **Student Feedback System** developed in this project provides a simple yet effective platform to collect, store, and analyze feedback from students. Built using Python and the Pandas library, the system allows for real-time interaction through a command-line interface, enabling users to add, view, search, and analyze feedback stored in a structured CSV file.

This project addresses a fundamental need in educational institutions — **gathering honest feedback** to evaluate teaching effectiveness, identify areas of improvement, and maintain academic standards. The feedback collected can be valuable for faculty development, curriculum enhancement, and institutional quality assurance.

In summary, the project has achieved its goal of creating a functional and interactive feedback system. It also provided hands-on experience in file handling, data analysis, exception handling, and user input validation in Python — strengthening core programming skills and project design capabilities.

# Refrences


1. Python Official Docs – https://docs.python.org/3/
2. Pandas Library Docs – https://pandas.pydata.org/docs/
3. W3Schools Python Tutorial – https://www.w3schools.com/python/
4. GeeksforGeeks – https://www.geeksforgeeks.org/python-programming-language/
5. Stack Overflow – https://stackoverflow.com/
6. Real Python – https://realpython.com/
7. Python CSV Module – https://docs.python.org/3/library/csv.html